HTTP 게이트웨이 구현 분석

개요

IPFS HTTP 게이트웨이는 IPFS 네트워크의 콘텐츠를 일반적인 웹 브라우저와 HTTP 클라이언트가 접근할 수 있도록 하는 HTTP-to-IPFS 브리지입니다. 게이트웨이는 IPFS 경로를 HTTP URL로 변환하고, 콘텐츠를 검색하여 표준 HTTP 응답으로 제공합니다.

게이트웨이 아키텍처

전체 구조

```
HTTP 요청 → 게이트웨이 핸들러 → 백엔드 → 블록 서비스 → 콘텐츠 검색 → HTTP 응답
```

주요 컴포넌트

1. 게이트웨이 옵션 (ServeOption)

```
func GatewayOption(paths ...string) ServeOption {
return func(n *core.IpfsNode, _ net.Listener, mux *http.ServeMux) (*http.ServeM
config, headers, err := getGatewayConfig(n)
if err != nil {
return nil, err
}
```

```
backend, err := newGatewayBackend(n)
if err != nil {
return nil, err
}

handler := gateway.NewHandler(config, backend)
handler = gateway.NewHeaders(headers).ApplyCors().Wrap(handler)
handler = otelhttp.NewHandler(handler, "Gateway")

for _, p := range paths {
mux.Handle(p+"/", handler)
}

return mux, nil
}
}
```

2. 호스트명 기반 게이트웨이

```
func HostnameOption() ServeOption {
return func(n *core.IpfsNode, _ net.Listener, mux *http.ServeMux) (*http.ServeM
// 호스트명 기반 라우팅 설정
handler = gateway.NewHostnameHandler(config, backend, childMux)
```

```
handler = gateway.NewHeaders(headers).ApplyCors().Wrap(handler)

mux.Handle("/", handler)

return  childMux, nil

}

}
```

게이트웨이 유형

1. 경로 기반 게이트웨이 (Path-based Gateway)

전통적인 IPFS 게이트웨이 방식으로, URL 경로에 IPFS 해시를 포함합니다.

URL 형식:

```
https://gateway.example.com/ipfs/QmHash...

https://gateway.example.com/ipns/example.com
```

특징:

- 간단한 구현
- 모든 콘텐츠가 동일한 오리진 공유
- CORS 제한 존재

2. 서브도메인 게이트웨이 (Subdomain Gateway)

각 IPFS 리소스를 별도의 서브도메인으로 제공하여 오리진 격리를 제공합니다.

URL 형식:

```
https://QmHash....ipfs.gateway.example.com/

https://example-com.ipns.gateway.example.com/
```

특징:

- 오리진 격리 제공
- 보안 향상
- 웹 표준 준수

```
var  subdomainGatewaySpec = &gateway.PublicGateway{

Paths: defaultPaths,

UseSubdomains: true,

}
```

3. 신뢰할 수 없는 게이트웨이 (Trustless Gateway)

검증 가능한 원시 데이터만 제공하는 게이트웨이입니다.

특징:

- 역직렬화된 응답 비활성화
- CAR 파일 형식으로 응답
- 클라이언트 측 검증 필요

```
gwConfig := gateway.Config{

DeserializedResponses: false, // Trustless-only

NoDNSLink: true, // No DNS resolution
```

```
}
```

콘텐츠 검색 과정

1. 경로 해석

```
func (o *offlineGatewayErrWrapper) ResolvePath(ctx context.Context, path path.Im
md, err := o.gwimpl.ResolvePath(ctx, path)
err = offlineErrWrap(err)
return  md, err
}
```

단계:

1. **경로 파싱**: `/ipfs/QmHash/path/to/file` → CID + 경로
2. **IPNS 해석**: IPNS 이름을 IPFS 경로로 변환
3. **DNSLink 해석**: DNS TXT 레코드를 통한 도메인 해석
4. **경로 검증**: 유효한 IPFS 경로인지 확인

2. 블록 검색

```
func  newGatewayBackend(n *core.IpfsNode) (gateway.IPFSBackend, error) {
cfg, err := n.Repo.Config()
if  err != nil {
return  nil, err
```

```
}

bserv := n.Blocks
var  vsRouting routing.ValueStore = n.Routing
nsys := n.Namesys
pathResolver := n.UnixFSPathResolver

if  cfg.Gateway.NoFetch {
// 오프라인 모드: 로컬 블록만 사용
bserv = blockservice.New(bserv.Blockstore(), offline.Exchange(bserv.Blockstore()
vsRouting = offlineroute.NewOfflineRouter(n.Repo.Datastore(), n.RecordValidator)
pathResolver = n.OfflineUnixFSPathResolver
}

backend, err := gateway.NewBlocksBackend(bserv,
gateway.WithValueStore(vsRouting),
gateway.WithNameSystem(nsys),
gateway.WithResolver(pathResolver),
)
return &offlineGatewayErrWrapper{gwimpl: backend}, nil
}
```

3. 콘텐츠 유형별 처리

파일 검색

```
func (o *offlineGatewayErrWrapper) Get(ctx context.Context, path path.ImmutableP
md, n, err := o.gwimpl.Get(ctx, path, ranges...)
err = offlineErrWrap(err)
return  md, n, err
}
```

디렉토리 검색

```
func (o *offlineGatewayErrWrapper) GetAll(ctx context.Context, path path.Immutab
md, n, err := o.gwimpl.GetAll(ctx, path)
err = offlineErrWrap(err)
return  md, n, err
}
```

블록 검색

```
func (o *offlineGatewayErrWrapper) GetBlock(ctx context.Context, path path.Immut
md, n, err := o.gwimpl.GetBlock(ctx, path)
err = offlineErrWrap(err)
return  md, n, err
}
```

게이트웨이 설정

기본 설정

```
func getGatewayConfig(n *core.IpfsNode) (gateway.Config, map[string][]string, e
cfg, err := n.Repo.Config()
if err != nil {
return gateway.Config{}, nil, err
}

gwCfg := gateway.Config{
DeserializedResponses: cfg.Gateway.DeserializedResponses.WithDefault(config.Defa
DisableHTMLErrors: cfg.Gateway.DisableHTMLErrors.WithDefault(config.DefaultDisab
NoDNSLink: cfg.Gateway.NoDNSLink,
PublicGateways: map[string]*gateway.PublicGateway{},
RetrievalTimeout: cfg.Gateway.RetrievalTimeout.WithDefault(config.DefaultRetriev
MaxConcurrentRequests: int(cfg.Gateway.MaxConcurrentRequests.WithDefault(int64(c
}

return gwCfg, cfg.Gateway.HTTPHeaders, nil
}
```

공개 게이트웨이 설정

```
type GatewaySpec struct {
Paths []string // 처리할 경로 접두사
UseSubdomains bool // 서브도메인 사용 여부
NoDNSLink bool // DNSLink 비활성화
InlineDNSLink Flag // DNSLink 인라인 처리
DeserializedResponses Flag // 역직렬화 응답 허용
}
```

설정 예시:

```
{
"Gateway": {
"PublicGateways": {
"localhost": {
"Paths": ["/ipfs", "/ipns"],
"UseSubdomains": true
},
"example.com": {
"Paths": ["/ipfs"],
"UseSubdomains": false,
"NoDNSLink": true
}
}
```

```
  }

}
```

신뢰할 수 있는 vs 신뢰할 수 없는 게이트웨이

신뢰할 수 있는 게이트웨이 (Trusted Gateway)

특징:

- 콘텐츠를 역직렬화하여 제공
- HTML, 이미지, 비디오 등을 브라우저에서 직접 렌더링
- 사용자 편의성 높음
- 게이트웨이 신뢰 필요

사용 사례:

- 일반 웹 브라우징
- 콘텐츠 미리보기
- 개발 및 테스트

신뢰할 수 없는 게이트웨이 (Trustless Gateway)

특징:

- 원시 블록 데이터만 제공
- CAR (Content Addressable aRchive) 형식
- 클라이언트 측 검증 가능
- 게이트웨이 신뢰 불필요

사용 사례:

- 분산 애플리케이션
- 데이터 무결성이 중요한 경우
- 검열 저항성이 필요한 경우

```
func Libp2pGatewayOption() ServeOption {
return func(n *core.IpfsNode, _ net.Listener, mux *http.ServeMux) (*http.ServeM
gwConfig := gateway.Config{
DeserializedResponses: false, // Trustless-only
NoDNSLink: true, // No DNS resolution
PublicGateways: nil,
Menu: nil,
}
handler := gateway.NewHandler(gwConfig, &offlineGatewayErrWrapper{gwimpl: backen
mux.Handle("/ipfs/", handler)
return mux, nil
}
}
```

성능 최적화 및 캐싱

1. 동시 요청 제한

```
MaxConcurrentRequests: int(cfg.Gateway.MaxConcurrentRequests.WithDefault(int64(c
```

과도한 동시 요청을 제한하여 서버 리소스를 보호합니다.

2. 검색 타임아웃

```
RetrievalTimeout: cfg.Gateway.RetrievalTimeout.WithDefault(config.DefaultRetriev
```

콘텐츠 검색에 대한 타임아웃을 설정하여 무한 대기를 방지합니다.

3. 로컬 캐싱

```
func (o *offlineGatewayErrWrapper) IsCached(ctx context.Context, path path.Path)
return  o.gwimpl.IsCached(ctx, path)
}
```

자주 요청되는 콘텐츠를 로컬에 캐시하여 응답 속도를 향상시킵니다.

4. NoFetch 모드

```
if  cfg.Gateway.NoFetch {
bserv = blockservice.New(bserv.Blockstore(), offline.Exchange(bserv.Blockstore()
pathResolver = n.OfflineUnixFSPathResolver
}
```

네트워크에서 블록을 가져오지 않고 로컬 블록만 사용하는 모드입니다.

에러 처리

오프라인 에러 래핑

```
func offlineErrWrap(err error) error {
if errors.Is(err, iface.ErrOffline) {
return fmt.Errorf("%s : %w", err.Error(), gateway.ErrServiceUnavailable)
}
return err
}
```

오프라인 상태에서 발생하는 에러를 적절한 HTTP 상태 코드로 변환합니다.

HTTP 상태 코드 매핑

- **200 OK**: 성공적인 콘텐츠 검색
- **404 Not Found**: 존재하지 않는 콘텐츠
- **429 Too Many Requests**: 동시 요청 제한 초과
- **504 Gateway Timeout**: 검색 타임아웃
- **503 Service Unavailable**: 서비스 이용 불가

보안 고려사항

1. 오리진 격리

서브도메인 게이트웨이를 통해 각 IPFS 리소스를 별도의 오리진으로 격리합니다.

2. CORS 설정

```
handler = gateway.NewHeaders(headers).ApplyCors().Wrap(handler)
```

적절한 CORS 헤더를 설정하여 크로스 오리진 요청을 제어합니다.

3. DNSLink 보안

```
NoDNSLink: cfg.Gateway.NoDNSLink
```

DNSLink 해석을 비활성화하여 DNS 기반 공격을 방지할 수 있습니다.

4. 콘텐츠 검증

신뢰할 수 없는 게이트웨이 모드에서는 클라이언트가 콘텐츠를 직접 검증할 수 있습니다.

모니터링 및 로깅

OpenTelemetry 통합

```
handler = otelhttp.NewHandler(handler, "Gateway")
```

모든 게이트웨이 요청에 대한 추적 정보를 수집합니다.

메트릭 수집

- 요청 수 및 응답 시간
- 에러율 및 에러 유형
- 캐시 히트율

- 동시 연결 수

이러한 포괄적인 게이트웨이 시스템을 통해 IPFS Kubo는 웹 브라우저와 기존 HTTP 클라이언트가 IPFS 네트워크의 콘텐츠에 쉽게 접근할 수 있도록 하면서도, 보안과 성능을 모두 고려한 다양한 게이트웨이 모드를 제공합니다.